

Operator Learning for Parametric Darcy Flow

Fourier Neural Operator vs. MLP Baseline — Stage 3

Team PINNacles
Aryamann Srivastava

AE646: Scientific Machine Learning for Fluid Mechanics
Department of Aerospace Engineering, IIT Kanpur

November 2026

Problem

2D parametric Darcy flow (steady pressure in a porous medium):

$$-\nabla \cdot (\kappa(x, y) \nabla u(x, y)) = f, \quad (x, y) \in (0, 1)^2$$

$$u = 0 \text{ on } \partial\Omega, \quad f = 1$$

- $\kappa \in \{0.1, 1.0\}$: piecewise-constant permeability (thresholded Gaussian random field).
- Goal: learn the solution operator $\mathcal{G} : \kappa \mapsto u$ once, then evaluate in milliseconds.

Why it matters. Classical solvers (FDM/FEM) re-solve the full PDE for every new κ ; UQ / optimisation / control need thousands of solves.

Theme #8: operator learning for parametric PDEs. Baseline = MLP; SciML model = FNO.

Dataset — real PDEBench 2D Darcy ($\beta = 1.0$)

- Official source: DaRUS / Uni Stuttgart, DOI 10.18419/darus-2986; checksum-verified download (`src/download_data.py`). 10 000 samples at native 128×128 .
- Reproducible (seed 42), non-overlapping subset: **900 train / 100 validation / 200 test** — matching the sample counts used by Li et al. (2021).
- Main pipeline at 64×64 : subsample every second grid point. Normalisation uses training statistics only.
- The 200 test samples are *also* kept at native 128×128 (`test_hires.npz`) — untouched real ground truth for the zero-shot super-resolution check.

MLP baseline

- Flatten $64 \times 64 \times 3 \rightarrow 3 \times 2048$ (GELU) $\rightarrow 64 \times 64$.
- 42.0 M parameters. No spatial inductive bias; input size hard-locked to 64×64 .

Fourier Neural Operator

- Lift $3 \rightarrow 64$; $4 \times [\text{SpectralConv} (12 \text{ modes}) + 1 \times 1 \text{ conv} + \text{LayerNorm} + \text{GELU}]$; project $64 \rightarrow 128 \rightarrow 1$.
- Spectral weights live in Fourier space $\Rightarrow$ resolution-invariant.
- Original: 4.7 M params. Improved: width 128, 20 modes, 6 blocks $\Rightarrow$ 78.8 M.

Training: AdamW (peak 10^{-3} , per-step cosine cycling to 0), batch 16, weight decay 10^{-4} , MSE loss on standardised pressure. 100 epochs (200 for FNO improved); the lowest-validation-error checkpoint is kept (no early stopping). Metric reported in physical (denormalised) units.

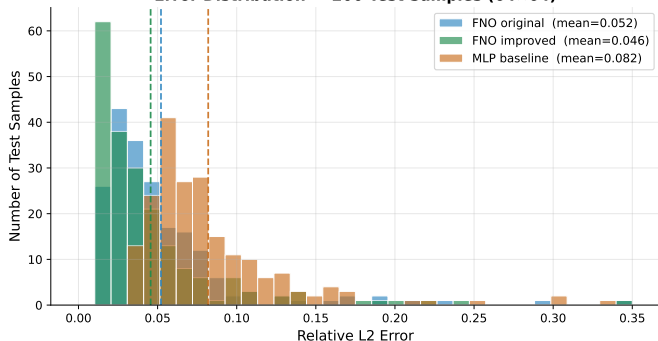
Test-set accuracy (64×64 , 200 held-out samples)

Model	Params	Mean	Median	Std	Min	Max
MLP baseline	42.0 M	0.0820	0.0695	0.0456	0.0325	0.3367
FNO original	4.7 M	0.0521	0.0398	0.0451	0.0155	0.3467
FNO improved	78.8 M	0.0456	0.0307	0.0487	0.0106	0.3535

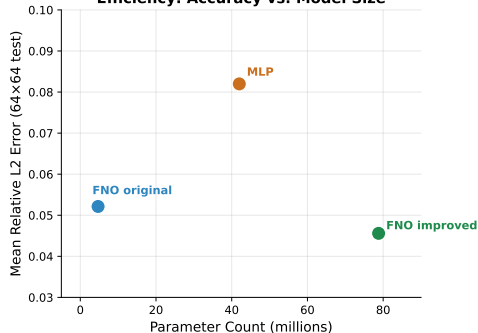
- FNO original: **37% lower mean error than MLP with $9\times$ fewer parameters.**
- Scaling to 78.8 M ($17\times$) buys only a further 12% — the 12-mode truncation, not channel count, is the limit.
- Similar std across models (0.045–0.049): the hard cases (complex multi-region κ) are hard for every architecture.

Error distribution and parameter efficiency

Error Distribution — 200 Test Samples (64×64)

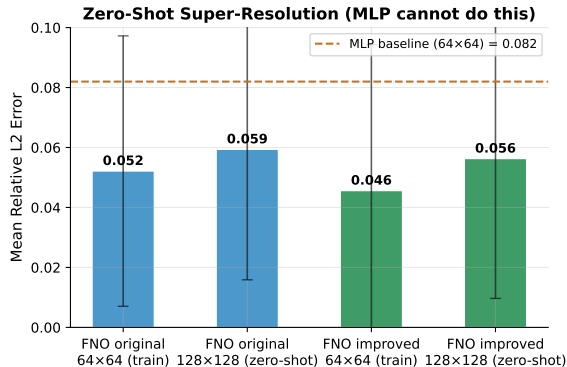


Efficiency: Accuracy vs. Model Size



Right-skewed errors: most samples well below 0.1, a $\lesssim 5\%$ tail with highly heterogeneous κ drives the maximum toward 0.35. FNO sits in the favourable low-parameter / low-error corner.

Zero-shot super-resolution (evaluate at native 128×128)

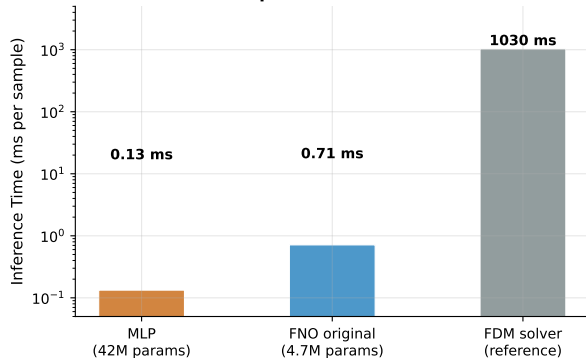


Model	64^2 (train)	128^2 (zero-shot)
FNO original	0.0521	0.0594
FNO improved	0.0456	0.0563

Same weights, real native-resolution κ , *no* retraining. Error rises only +14% (original): Darcy solutions are low-frequency dominated, so the unmodelled high modes carry little energy. The MLP cannot run this test at all (fixed input size).

Inference speed

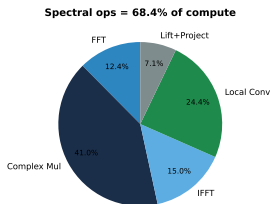
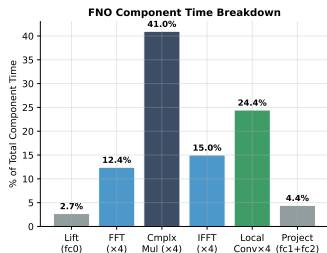
Inference Speed vs. Traditional Solver



Solver	ms/sample	vs. FDM
FDM (SciPy sparse, CPU)	1030.1	1×
MLP baseline (GPU)	0.13	7740×
FNO original (GPU)	0.71	1443×

Both surrogates are $> 1000\times$ faster than the numerical solver. MLP is $\sim 5.4\times$ faster than FNO per sample (FFT latency) — but FNO's accuracy makes it the better operator surrogate.

Where does FNO spend its time?



Layer-wise profiling with a device sync before every timer read (measured on Apple Silicon / MPS, 500 iterations).

- Spectral ops (FFT + complex multiply + IFFT): **68.4%**.
- Complex weight multiply alone: **41.0%** — a batched einsum over $64 \times 64 \times 12 \times 12$ complex tensors.

Sync-per-timer inflates absolute times (≈ 19.6 ms) far above the fused 0.71 ms; the *relative* split is the portable result. Optimisation target: low-rank / Tucker-decomposed spectral weights.

- Darcy pressure u solves an elliptic PDE: globally smooth, low-wavenumber dominated. FNO's 12-mode truncation *is* this prior — it keeps the coefficients that carry the energy.
- MLP flattening destroys locality in the first layer; its 42 M-parameter map is poorly regularised for the task and must learn spatial correlations by brute force.
- High-error samples ($\epsilon > 0.2$): many disconnected κ regions with sharp interfaces $\Rightarrow$ rapid transitions in ∇u needing high modes. Adding modes/depth helps the easy and typical cases (median $0.0398 \rightarrow 0.0307$) but barely moves the worst case (0.347 vs. 0.354).

Conclusion and reproducibility

- FNO beats the MLP baseline: **37% lower error**, $9\times$ **fewer parameters**, zero-shot to $2\times$ resolution (+14%), $> 1000\times$ faster than FDM.
- The spectral inductive bias — “elliptic PDE solutions are smooth” — is nearly exact for Darcy flow, which explains the gap with the unbiased MLP.
- Reproducible: seed 42 throughout; pytest suite (16 tests); metrics/figures regenerate from committed JSON; MPS re-run of the pipeline matched the GPU numbers (FNO 0.0521 vs. 0.0544).

AI-tool declaration. Claude Code assisted with debugging (the GPU timing fix), LaTeX formatting, and slide-build scripting. All results are from experiments run on real hardware; every number, figure, and claim was reviewed and is owned by the author. Full statement: `CONTRIBUTION_STATEMENT.pdf`.

- ① Z. Li et al., “Fourier Neural Operator for Parametric Partial Differential Equations,” ICLR 2021.
- ② M. Takamoto et al., “PDEBench: An Extensive Benchmark for Scientific Machine Learning,” NeurIPS 2022. DOI [10.18419/darus-2986](https://doi.org/10.18419/darus-2986).
- ③ N. Kovachki et al., “Neural Operator: Learning Maps Between Function Spaces,” JMLR 2023.

Thank you — questions welcome.